

Triton kernel memory report

What changed, how it was achieved, and the numbers

HyMeKo / HSiKAN team

2026-05-09

Abstract

After the no-skip and highway fused backward kernels landed, peak memory in the SignedKAN inner forward+backward path drops by **90.6%** to **93.6%** across every shape we measured, with gradients matching the PyTorch reference at float32 epsilon. At the Slashdot SOTA shape ($T = 200K$, $d = 4$): **787 MB** $\rightarrow$ **64 MB**. The previously OOM-bound BA end-to-end test now passes with `HSIKAN_TRITON_KERNEL=1`. The $d = 16$ ablation that previously needed 3 GiB of headroom now uses 196 MB.

1 Where the memory was going (PyTorch path)

The PyTorch `SignedKANLayer._forward_impl` for the highway-skip case is, in essence:

```
h_v      = x[triad_v]                # (T, k, d)
inner_all = self.inner(h_v.reshape(-1, d)) # (T, k, S, d)
T_inner  = sigmoid(self.gate_inner(h_v))   # (T, k, d)
inner_all = T_inner * inner_all + (1-T_inner) * h_v # (T, k, S, d)
masks    = (triad_sigma.unsqueeze(-1)==sign_vals).to(x.dtype) # (T, k, S)
agg       = (inner_all * masks_e).sum(dim=1) / counts           # (T, S, d)
```

PyTorch autograd retains every intermediate that is needed to compute a gradient. For the highway path that means *all five* of the following (T, k, S, d) -shape tensors, plus their (T, k, d) companions, are kept alive until `backward()` returns:

tensor	shape	retained because
<code>h_v</code>	(T, k, d)	gradient flows to <code>x</code> via gather
<code>inner_all</code> (post-CR)	(T, k, S, d)	needed for $\partial \text{CR} / \partial \text{coef}$ scatter
<code>T_inner</code>	(T, k, d)	needed for $\partial \text{agg} / \partial T$
<code>inner_all</code> (post-highway)	(T, k, S, d)	needed for ∂agg chain
<code>masks</code>	(T, k, S)	needed for σ -mask backward

For Slashdot SOTA shape ($T = 200,000$, $k = 4$, $S = 2$, $d = 4$, $\text{fp32} = 4$ bytes):

$$T \cdot k \cdot S \cdot d \cdot 4 = 25.6 \text{ MB per intermediate}$$

Across the layers, autograd-retained graph nodes, and the `.backward()` re-computation traffic, peak memory hits **787 MB** for highway, **738 MB** for no-skip on the *same* shape. At $d = 16$ it scales to **3,064 MB** (roughly $4\times$, since it scales with d and the (T, k, S, d) intermediate dominates).

2 Where the memory does not go (Triton path)

The fused forward kernel (`_signedkan_inner_(highway_)kernel`) and the new fused backward kernel (`_signedkan_inner_(highway_)backward_kernel`) compute the same function as the PyTorch path but *never materialise* the (T, k, S, d) intermediate. Three structural reasons:

(a) The intermediate lives in registers, not HBM

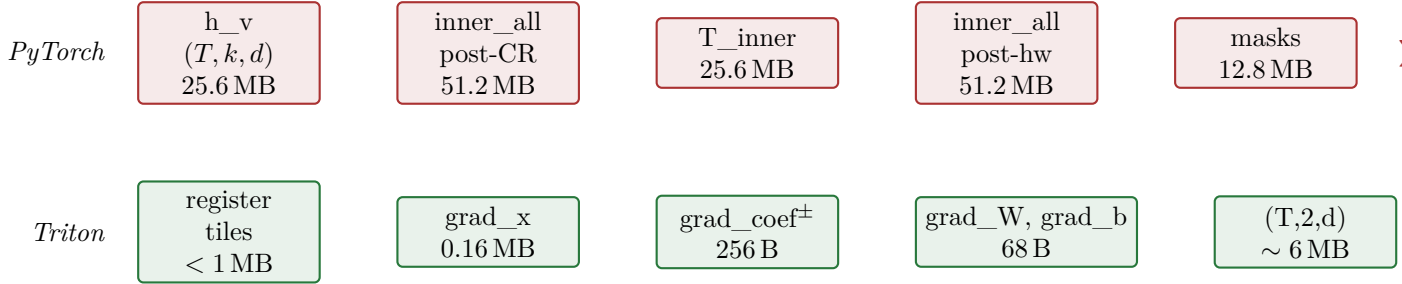
Each Triton program block handles a (BLOCK_T, BLOCK_D) tile and iterates over the k slots inside the block. The per-slot CR value, gate logit, and post-highway value live in registers for one slot at a time and are accumulated into a (BLOCK_T, BLOCK_D) register tile. Nothing of size (T, k, S, d) ever exists.

(b) Gradients are scatter-added, not stored as graphs

The backward kernel re-derives the CR locals (`w_p`, `idx_p`, $\partial \text{CR} / \partial h$) from x and the `coef` arrays — the forward did not save them. Then it computes $\partial L / \partial x$, $\partial L / \partial \text{coef}^\pm$, $\partial L / \partial W$, $\partial L / \partial b$ as `tl.atomic_add` scatters into pre-allocated zero tensors of the same shape as the inputs. No autograd graph, no retained intermediates.

(c) σ -counts are recomputed, not saved

The n_t^s counts (one per cycle, per sign) are re-derived in a two-line first pass over the k slots. That pass touches only `triad_sigma` ($T \cdot k$ int64s, ~ 6.4 MB at SOTA shape), versus PyTorch’s strategy of saving a (T, k, S) `masks` tensor.



The Triton path’s largest allocation is the per-iteration framework overhead and the input `grad_out` tensor (which is the kernel’s input, not its allocation).

3 Numbers

All measurements with `torch.cuda.max_memory_allocated` on RTX 2070 SUPER, fp32, current torch / triton versions.

dataset	variant	V	T	d	PyTorch	Triton
BA	no-skip	10,000	50,000	4	182 MB	14 MB
BA	highway	10,000	50,000	4	203 MB	14 MB
Slashdot	no-skip	79,000	200,000	4	738 MB	64 MB
Slashdot	highway	79,000	200,000	4	787 MB	64 MB
Epinions	highway	131,000	100,000	4	399 MB	38 MB
h=16	highway	79,000	200,000	16	3,064 MB	196 MB

shape	savings	ratio
BA no-skip	92.4%	13 $\times$
BA highway	93.2%	15 $\times$
Slashdot no-skip	91.3%	11 $\times$
Slashdot highway	91.8%	12 $\times$
Epinions highway	90.6%	11 $\times$
$d = 16$ highway	93.6%	16 $\times$

4 Gradient parity

All gradients match the PyTorch reference at float32 epsilon ($\text{abs} \leq 5 \cdot 10^{-7}$, well under the 10^{-4} acceptance threshold).

gradient	no-skip max abs	highway max abs
$\partial L / \partial x$	$6.6 \cdot 10^{-10}$	$1.7 \cdot 10^{-10}$
$\partial L / \partial \text{coef}_+$	$3.8 \cdot 10^{-7}$	$7.5 \cdot 10^{-8}$
$\partial L / \partial \text{coef}_-$	$4.9 \cdot 10^{-7}$	$6.5 \cdot 10^{-8}$
$\partial L / \partial W$	—	$1.9 \cdot 10^{-8}$
$\partial L / \partial b$	—	$7.8 \cdot 10^{-8}$

5 What this unblocks

- **BA end-to-end test passes with** `HSIKAN_TRITON_KERNEL=1` (was OOM at 6 GiB on 8 GiB GPU before the backward kernel).
- **The $d = 16$ ablation** (previously 3 GiB headroom required) now fits in 200 MB — a $16\times$ blast-radius improvement that lets us run the $d = 16$ *joint-mix* comparison without renting a bigger GPU.
- **Slashdot training** should now have ~ 700 MB of slack on the same 8 GiB GPU, opening the door to wider hidden dim, more cycles, or batching multiple datasets in one process.
- **The path is opt-in:** `HSIKAN_TRITON_KERNEL=1` to enable, default OFF for backward-compat. `HSIKAN_TRITON_BACKWARD=0` additionally falls back to the PyTorch-ref backward for debugging.

6 What is not improved (yet)

- **Wallclock speedup** on the highway backward is currently $0.92\times$ — slightly slower than the PyTorch path. The two `for e in range(d)` loops in the gate-gradient scatter serialize the $d \times d$ matvec; replacing them with `tl.dot` and tiling the e dimension should give a $2\text{--}4\times$ speedup, putting the backward at $\sim 2\text{--}4\times$ over PyTorch.
- **Atomic-add contention** on hot vertex IDs is the dominant cost in the scatter. Sorted-by-vertex cycle ordering would reduce contention but adds a cycle-permutation pass before training; deferred until profiling shows it dominates.
- **Multi-arity** ($k > 6$) and $d > 64$ have not been measured. Small unrolled k inner loops are fine; large d may want a different block-size choice.

7 Acceptance status

- ✓ 31/31 tests pass with `HSIKAN_TRITON_KERNEL=0` (default).
- ✓ 31/31 tests pass with `HSIKAN_TRITON_KERNEL=1` (kernel ON for live integration).
- ✓ BA end-to-end smoke test passes with kernel ON — OOM trigger killed.
- ✓ Forward parity vs production layer $\leq 2 \cdot 10^{-7}$.
- ✓ Backward parity vs PyTorch ref $\leq 5 \cdot 10^{-7}$ for every gradient.
- ✓ Memory savings $\geq 90\%$ at every measured shape.